



PROTOCOLO DE INGENIERÍA & CONTROL DE VERSIONES

ESTRATEGIA Y PROTOCOLO DE CONTROL DE VERSIONES (GIT WORKFLOW)

Convención de Ramas, Reglas de Pull Request y Estándar Conventional Commits — Normas APA (7.ª Ed.)

LÍDER DE ARQUITECTURA & QA LEAD	FECHA Y VERSIÓN
Alan Martinez	2026-08-21 Versión 1.0 (Estrategia de Versionado Aprobada)
EQUIPO DE DESARROLLO	MODELO DE TRABAJO GIT
Ivan Ismael Cardozo, Alex Heredia, Marcos Silvani, Franco Sarraute	Gitflow Adaptado + Conventional Commits + Code Review Mandatory

1. Propósito y Modelo de Trabajo

El presente documento fija el protocolo obligatorio de gobernanza de código fuente para el repositorio del proyecto Plataforma Hospitalaria de Código Azul. Se adopta un modelo basado en **Gitflow Adaptado** para garantizar la estabilidad del código en producción y la integración continua fluida entre los desarrolladores.

2. Estructura y Convención de Nombres para Ramas

Queda prohibido realizar commits directos sobre las ramas principales. Todo cambio debe realizarse a través de ramas auxiliares debidamente nombradas:

Tabla 1
Tipología de Ramas y Políticas de Fusión

Tipo de Rama	Patrón de Nombre	Propósito y Rama de Destino
Main (Producción)	main	Código estable y desplegable en producción. Protegida contra pushes directos.
Development	develop	Rama de integración continua. Fusiona las funcionalidades listas antes del release.
Feature	feature/<modulo>-<descripcion>	Desarrollo de nuevas funcionalidades (ej. feature/auth-jwt-middleware). Nace de develop, integra a develop.
Bugfix	bugfix/<modulo>-<descripcion>	Corrección de errores encontrados en etapa de testing/QA. Nace de develop, integra a develop.
Hotfix	hotfix/<descripcion-critica>	Parche crítico de emergencia para fallos en producción. Nace de main, integra a main y develop.

Nota. Esquema estandarizado conforme a las mejores prácticas de ingeniería de software.

3. Convención de Commits (Conventional Commits v1.0.0)

Todos los mensajes de commit deben seguir estrictamente el estándar estandarizado para permitir la generación automática de changelogs:

```
<tipo>(<alcance/módulo>): <descripción corta en presente y minúsculas>
```

Ejemplos válidos en el repositorio:

- feat(auth): implementar middleware de validación de token JWT y roles RBAC
- fix(sockets): corregir reconexión automática en gateway de Socket.IO ante pérdida de red
- docs(uml): actualizar diagrama de secuencia con evento de confirmación ACK
- refactor(db): optimizar consulta de incidentes activos utilizando índice por estado

4. Reglas de Pull Requests (PR) y Revisión de Código

Para fusionar cambios en las ramas develop o main es obligatorio cumplir con las siguientes políticas de calidad:

- **Revisión de Código (Code Review):** Aprobación obligatoria por el Líder de Arquitectura (Alan Martinez) o un Revisor Designado.
- **Verificación Integración Continua (CI):** Ninguna PR se fusionará si rompe la compilación del proyecto o falla los tests automatizados.
- **Estrategia de Merge:** Estrategia 'Squash and Merge' obligatoria para mantener un historial limpio en develop y main.

5. Referencias Normativas (APA 7.ª Edición)

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress.

Conventional Commits. (2020). *Conventional Commits specification* (Versión 1.0.0). <https://www.conventionalcommits.org/>